

Semihosting for AArch32 and AArch64

2023Q3

Date of Issue: 6th October 2023

arm

1 Preamble

1.1 Abstract

This document describes the Arm® semihosting mechanism, and its extensions.

1.2 Keywords

Semihosting

1.3 Latest release and defects report

Please check [Application Binary Interface for the Arm® Architecture](#) for the latest release of this document.

Please report defects in this specification to the [issue tracker page on GitHub](#).

1.4 Licence

This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Grant of Patent License. Subject to the terms and conditions of this license (both the Public License and this Patent License), each Licensor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Licensed Material, where such license applies only to those patent claims licensable by such Licensor that are necessarily infringed by their contribution(s) alone or by combination of their contribution(s) with the Licensed Material to which such contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Licensed Material or a contribution incorporated within the Licensed Material constitutes direct or contributory patent infringement, then any licenses granted to You under this license for that Licensed Material shall terminate as of the date such litigation is filed.

1.5 About the license

As identified more fully in the [Licence](#) section, this project is licensed under CC-BY-SA-4.0 along with an additional patent license. The language in the additional patent license is largely identical to that in Apache-2.0 (specifically, Section 3 of Apache-2.0 as reflected at <https://www.apache.org/licenses/LICENSE-2.0>) with two exceptions.

First, several changes were made related to the defined terms so as to reflect the fact that such defined terms need to align with the terminology in CC-BY-SA-4.0 rather than Apache-2.0 (e.g., changing “Work” to “Licensed Material”).

Second, the defensive termination clause was changed such that the scope of defensive termination applies to “any licenses granted to You” (rather than “any patent licenses granted to You”). This change is intended to help maintain a healthy ecosystem by providing additional protection to the community against patent litigation claims.

1.6 Contributions

Contributions to this project are licensed under an inbound=outbound model such that any such contributions are licensed by the contributor under the same terms as those in the [Licence](#) section.

1.7 Trademark notice

The text of and illustrations in this document are licensed by Arm under a Creative Commons Attribution–Share Alike 4.0 International license (“CC-BY-SA-4.0”), with an additional clause on patents. The Arm trademarks featured here are registered trademarks or trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. All rights reserved. Please visit <https://www.arm.com/company/policies/trademarks> for more information about Arm’s trademarks.

1.8 Copyright

Copyright (c) 2016, 2020-2023, Arm Limited and its affiliates. All rights reserved.

Contents

1	Preamble	2
1.1	Abstract	2
1.2	Keywords	2
1.3	Latest release and defects report	2
1.4	Licence	3
1.5	About the license	3
1.6	Contributions	3
1.7	Trademark notice	3
1.8	Copyright	3
2	About this document	6
2.1	Change control	6
2.2	Terms and abbreviations	6
3	Introduction	8
3.1	Related information	9
4	The semihosting interface	10
4.1	Related information	11
5	Semihosting extensions	12
5.1	Semihosting feature bit reporting sequence format	12
5.2	Requirements for semihosting implementations	12
5.3	Requirements for semihosting callers	13
5.4	Example pseudocode function for querying feature bits	13
5.5	Semihosting extensions available	14
6	Semihosting operations	15
6.1	SYS_CLOCK (0x10)	16
6.2	SYS_CLOSE (0x02)	17
6.3	SYS_ELAPSED (0x30)	18
6.4	SYS_ERRNO (0x13)	19
6.5	SYS_EXIT (0x18)	20
6.6	SYS_EXIT_EXTENDED (0x20)	22
6.7	SYS_FLEN (0x0C)	23
6.8	SYS_GET_CMDLINE (0x15)	24
6.9	SYS_HEAPINFO (0x16)	25
6.10	SYS_ISERROR (0x08)	26
6.11	SYS_ISTTY (0x09)	27
6.12	SYS_OPEN (0x01)	28
6.13	SYS_READ (0x06)	30
6.14	SYS_READC (0x07)	31

6.15	SYS_REMOVE (0x0E)	32
6.16	SYS_RENAME (0x0F)	33
6.17	SYS_SEEK (0x0A)	34
6.18	SYS_SYSTEM (0x12)	35
6.19	SYS_TICKFREQ (0x31)	36
6.20	SYS_TIME (0x11)	37
6.21	SYS_TMPNAM (0x0D)	38
6.22	SYS_WRITE (0x05)	39
6.23	SYS_WRITEC (0x03)	40
6.24	SYS_WRITE0 (0x04)	41

2 About this document

2.1 Change control

2.1.1 Current status and anticipated changes

The following support level definitions are used by the Arm ABI specifications:

Release

Arm considers this specification to have enough implementations, which have received sufficient testing, to verify that it is correct. The details of these criteria are dependent on the scale and complexity of the change over previous versions: small, simple changes might only require one implementation, but more complex changes require multiple independent implementations, which have been rigorously tested for cross-compatibility. Arm anticipates that future changes to this specification will be limited to typographical corrections, clarifications and compatible extensions.

Beta

Arm considers this specification to be complete, but existing implementations do not meet the requirements for confidence in its release quality. Arm may need to make incompatible changes if issues emerge from its implementation.

Alpha

The content of this specification is a draft, and Arm considers the likelihood of future incompatible changes to be significant.

All content in this document is at the **Release** quality level.

2.1.2 Change history

If there is no entry in the change history table for a release, there are no changes to the content of the document for that release.

Issue	Date	Change
2.0	16 December 2016	Semihosting extensions incorporated and text updated
2019Q4	30 January 2020	In the <i>Requirements for semihosting callers</i> example, change <code>_</code> to <code>-</code> in <code>:semihosting-features</code> .
2020Q4	21 st December 2020	- document released on Github - new Licence: CC-BY-SA-4.0 - new sections on Contributions, Trademark notice, and Copyright
2023Q3	6 st October 2023	Specify SYS_TIME return value as unsigned

2.2 Terms and abbreviations

The ABI for the Arm Architecture uses the following terms and abbreviations.

AAPCS

Procedure Call Standard for the Arm Architecture.

ABI

Application Binary Interface:

1. The specifications to which an executable must conform in order to execute in a specific execution environment. For example, the Linux ABI for the Arm Architecture.
2. A particular aspect of the specifications to which independently produced relocatable files must conform in order to be statically linkable and executable. For example, the [AAELF32](#), [RTABI32](#), ...

AEABI

(Embedded) ABI for the Arm architecture (this ABI...)

Arm-based

... based on the Arm architecture ...

core registers

The general purpose registers visible in the Arm architecture's programmer's model, typically r0-r12, SP, LR, PC, and CPSR.

EABI

An ABI suited to the needs of embedded, and deeply embedded (sometimes called free standing), applications.

Q-o-I

Quality of Implementation – a quality, behavior, functionality, or mechanism not required by this standard, but which might be provided by systems conforming to it. Q-o-I is often used to describe the toolchain-specific means by which a standard requirement is met.

VFP

The Arm architecture's Floating Point architecture and instruction set. In this ABI, this abbreviation includes all floating point variants regardless of whether or not vector (V) mode is supported.

3 Introduction

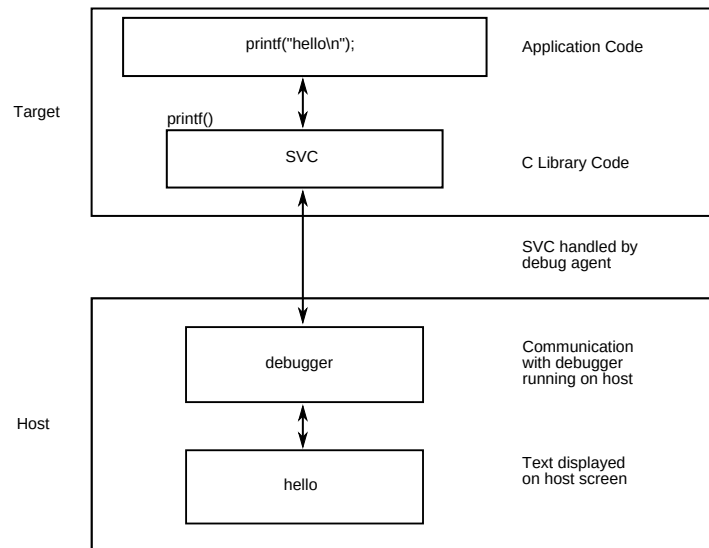
Semihosting is a mechanism that enables code running on an ARM target or emulator to communicate with and use the Input/Output facilities on a host computer. The host must be running the emulator, or a debugger that is attached to the ARM target.

Examples of these facilities include keyboard input, screen output, and disk I/O. For example, you can use this mechanism to enable functions in the C library, such as `printf()` and `scanf()`, to use the screen and keyboard of the host instead of having a screen and keyboard on the target system.

This is useful because development hardware often does not have all the input and output facilities of the final system. Semihosting enables the host computer to provide these facilities.

Semihosting is implemented by a set of defined software instructions that generate exceptions from program control. The application invokes the appropriate semihosting call and the debug agent then handles the exception. The debug agent provides the required communication with the host.

The semihosting interface is common across all debug agents that are provided by ARM. Semihosted operations work when you are debugging applications on your development platform, as shown in the following figure:



In many cases, semihosting is invoked by code within library functions. The application can also invoke the semihosting operation directly.

Note

The instruction that is used to make semihosting calls can be SVC, HLT, or BKPT, depending on the processor. See [The semihosting interface](#) for more detail.

The following terms are used in the semihosting documentation:

Semihosting Implementation

An agent (for example, a debugger or emulator) that services semihosting operation requests from a Semihosting Caller executing on an ARM target.

Semihosting Caller

A program executing on an ARM target that sends requests to a Semihosting Implementation (for example, a debugger or emulator) to service semihosting operations.

3.1 Related information

- [The semihosting interface](#)
- [The ARM C and C++ libraries](#)

4 The semihosting interface

Semihosting is supported for ARM A and R profiles using the A64, A32, and T32 instruction sets, and for M profile using the T32 instruction set.

Semihosting operations are requested using a trap instruction, which is a software instruction that generates exceptions from program control. Semihosting callers issue trap instructions, and the semihosting implementation then handles the resulting exception to perform the required semihosting operation. The trap instruction can be SVC, HLT, or BKPT, as indicated in the *Semihosting Trap Instructions and Encodings* table:

Semihosting Trap Instructions and Encodings

Profile	Instruction Set	Instruction	Opcode
A+R Profile	A64	HLT #0xF000	0xD45E0000
		SVC #0x123456	0xEF123456
	A32	HLT #0xF000	0xE10F0070
		SVC #0xAB	0xDFAB
		HLT #0x3C	0xBABC
M--Profile	T32	BKPT #0xAB	0xBEAB

For A32 and T32 on A+R Profile, semihosting can use either an SVC or an HLT trap instruction. Semihosting implementations must support both trap instructions across all versions of the ARM architecture.

Note

This requirement includes supporting the HLT encodings on ARMv7 and earlier processors, even though HLT is only defined as an instruction in ARMv8. This may require the semihosting implementation to trap the UNDEF exception.

The HLT encodings are new in version 2.0 of the semihosting specification. Where possible, have semihosting callers continue to use the previously existing trap instructions to ensure compatibility with legacy semihosting implementations. These trap instructions are HLT for A64, SVC on A+R profile A32 or T32, and BKPT on M profile. However, it is necessary to change from SVC to HLT instructions to support AArch32 semihosting properly in a mixed AArch32/AArch64 system.

Note

ARM encourages semihosting callers to implement support for trapping using HLT on A32 and T32 as a configurable option. ARM strongly discourages semihosting callers from mixing the HLT and SVC mechanisms within the same executable.

The OPERATION NUMBER REGISTER must contain the number of the semihosting operation to be performed. This number is given in parentheses after the operation name in the following sections. For example, [SYS_OPEN \(0x01\)](#).

Parameters are passed to the operation using the PARAMETER REGISTER. For most operations, the PARAMETER REGISTER must contain a pointer to a data block that holds the parameters. In some cases a single parameter is passed directly in the PARAMETER REGISTER, or no parameters are passed at all.

Results are returned in the RETURN REGISTER, either as an explicit return value or as a pointer to a data block. If no result is returned, assume that the RETURN REGISTER is corrupted. Some operations also return information in the PARAMETER REGISTER.

Multi-byte values in memory must be formatted as pure little-endian or pure big-endian to match the endianness mapping configuration of the processor.

The *Registers and field size* table shows the specific registers that are used, and the size of the fields in the data block, which depend on whether the caller is 32-bit or 64-bit.

Registers and field size

	32-bit	64-bit
OPERATION NUMBER REGISTER	R0	W0
PARAMETER REGISTER	R1	X1
RETURN REGISTER	R0	X0
Data block field size	32 bits	64 bits

Note

The operation number is passed in W0 for the 64-bit ABI, which is the bottom 32 bits of the 64-bit register X0. Semihosting implementations must not assume that the top 32 bits of X0 are 0.

A few semihosting operations have other differences between the 32-bit and 64-bit versions. These differences are described in the documentation of those operations.

The available semihosting operation numbers are allocated as follows:

0x00-0x31

Used by ARM.

0x32-0xFF

Reserved for future use by ARM.

0x100-0x1FF

Reserved for user applications. These semihosting operation numbers are not used by ARM. However if you are writing your own SVC operations, you are advised to use a different SVC number, rather than using the semihosted SVC number and these operation type numbers.

0x200-0xFFFFFFFF

Undefined and currently unused. ARM recommends that you do not use these semihosting operation numbers.

Note

Previous versions of the semihosting specification included the operation numbers 0x17 (angel_SWIreason_EnterSVC) and 0x19 (angelSWI_Reason_SyncCacheRange). These semihosting operation numbers are now reserved, and are not to be used or implemented.

4.1 Related information

- [A32 and T32 instruction sets](#)
- [A64 instruction set](#)

5 Semihosting extensions

The semihosting API provides an extension mechanism. Semihosting implementations can provide optional functionality and advertise to the semihosting caller that they do so. Semihosting callers must check that the implementation supports the optional functionality before attempting to use it.

Each extension that the specification defines has an associated feature bit. If the implementation provides the extension, then it reports the feature bit as 1. Each extension definition identifies the feature bit by which feature byte it is in, and which bit within that byte. For example, feature byte 0, bit 3. Feature bytes and bits within them are both numbered starting from 0, with feature bit 0 being the least significant bit. Extensions are independent. Unless otherwise stated, support for one feature does not imply support for any other feature. The caller must check the feature bit for every feature it wants to use.

5.1 Semihosting feature bit reporting sequence format

Feature bits are reported using a sequence of bytes, which are accessed by using the [SYS_OPEN \(0x01\)](#) call with the special path name `:semihosting-features`. The byte sequence has the following format:

```
byte 0: SHFB_MAGIC_0 0x53
byte 1: SHFB_MAGIC_1 0x48
byte 2: SHFB_MAGIC_2 0x46
byte 3: SHFB_MAGIC_3 0x42
byte 4: feature byte 0
byte 5: feature byte 1
byte 6: feature byte 2
...
```

There is no limit to the number of bytes that can be returned. As future extensions are defined, new bytes will be added. If the read reaches the end of the file before the byte that contains a particular feature bit, then that feature bit must be taken to be 0.

Since the file is a sequence of bytes, the order remains the same whether the processor is big-endian or little-endian. The contents must be treated as a byte array, not an array of words or any other larger type.

5.2 Requirements for semihosting implementations

An implementation that implements any semihosting extensions must do the following:

- Handle the special path name `:semihosting-features` in [SYS_OPEN \(0x01\)](#) when opened with mode 0 (r) or 1 (rb). Implement it to return a filehandle that behaves as if it were accessing a file containing a sequence of bytes in the format that is described in [feature-bits](#). Attempts to open this special path name with any other opening mode must fail. The implementation must support opening of this special path name multiple times, both consecutively and simultaneously.
- Support the following operations on the filehandle returned from [SYS_OPEN \(0x01\)](#) of `:semihosting-features`:

[SYS_FLEN \(0x0C\)](#)

Returns the length of the byte sequence, including the magic number bytes.

[SYS_ISTTY \(0x09\)](#)

Always returns 0.

[SYS_SEEK \(0x0A\)](#)

Permits random seeks to anywhere within the byte sequence.

[SYS_READ \(0x06\)](#)

Reads bytes from the sequence starting from the current seek position.

SYS_CLOSE (0x02)

Closes the filehandle.

The special path name does not correspond to a real file in a filesystem, and so no special case handling of it is expected for [SYS_REMOVE \(0x0E\)](#) or [SYS_RENAME \(0x0F\)](#).

5.3 Requirements for semihosting callers

To read the feature bits, the semihosting caller must:

1. Call [SYS_OPEN \(0x01\)](#) with the special filename `:semihosting-features` and the file opening mode `0 (r)`. If the [SYS_OPEN \(0x01\)](#) fails, then no extensions are supported, so all feature bits should be assumed to be 0.
2. Use the [SYS_READ \(0x06\)](#) call to read bytes from the filehandle returned by step 1. If the read returns fewer than 5 bytes, or the first 4 bytes are not the correct magic numbers, then no extensions are supported.

The caller must check all the magic bytes, but can optionally use [SYS_SEEK \(0x0A\)](#) to seek forwards in the byte sequence to the subsequent byte, or bytes, of interest. Seeking off the end of the file is undefined, so a caller wanting to use [SYS_SEEK \(0x0A\)](#) must first use [SYS_FLEN \(0x0C\)](#) to determine the size of the byte sequence.

3. If the [SYS_OPEN \(0x01\)](#) from step 1 did not fail, the caller must now use [SYS_CLOSE \(0x02\)](#) to close the filehandle.

It is important to follow this sequence to ensure correct behavior on legacy semihosting implementations, which do not recognize the special filename `:semihosting-features`.

5.4 Example pseudocode function for querying feature bits

The following C-like pseudocode describes one possible simple implementation of a check for a specific feature bit:

```
#define MAGICLEN 4
bool sh_feature_supported(int bytenum, int bitnum)
{
    unsigned char magic[MAGICLEN];
    unsigned char c;
    int fh;
    int len;

    fh = sys_open(":semihosting-features", 0);
    if (fh == -1) {
        return false;
    }
    len = sys_flen(fh);
    if (len <= bytenum) {
        sys_close(fh);
        return false;
    }
    if (sys_read(fh, &magic, MAGICLEN) != 0) {
        sys_close(fh);
        return false;
    }
    if (magic[0] != SHFB_MAGIC_0 ||
        magic[1] != SHFB_MAGIC_1 ||
        magic[2] != SHFB_MAGIC_2 ||
        magic[3] != SHFB_MAGIC_3) {
        sys_close(fh);
        return false;
    }
}
```

```

    }
    if (sys_seek(fh, bytenum) != 0) {
        sys_close(fh);
        return false;
    }
    if (sys_read(fh, &c, 1) != 0) {
        sys_close(fh);
        return false;
    }
    sys_close(fh);
    return (c & (1 << bitnum)) != 0;
}

```

More sophisticated implementations can choose to cache the contents of the start of the file with the magic bytes and the first few bytes of feature bit data. Conversely if, for instance, the caller knows in advance that it is only interested in feature bits inside the first feature byte, it can simplify this method to perform a single 5 byte read and need not call [SYS_FLEN \(0x0C\)](#) or [SYS_SEEK \(0x0A\)](#).

5.5 Semihosting extensions available

The *Semihosting Extensions Table* lists the semihosting extensions currently available, along with their associated feature byte and bit.

Semihosting Extensions

Name	Feature byte	Feature bit
SH_EXT_EXIT_EXTENDED	0	0
SH_EXT_STDOUT_STDERR	0	1

6 Semihosting operations

The following semihosting operations are available:

- `SYS_CLOCK` (0x10)
- `SYS_CLOSE` (0x02)
- `SYS_ELAPSED` (0x30)
- `SYS_ERRNO` (0x13)
- `SYS_EXIT` (0x18)
- `SYS_EXIT_EXTENDED` (0x20)
- `SYS_FLEN` (0x0c)
- `SYS_GET_CMDLINE` (0x15)
- `SYS_HEAPINFO` (0x16)
- `SYS_ISERROR` (0x08)
- `SYS_ISTTY` (0x09)
- `SYS_OPEN` (0x01)
- `SYS_READ` (0x06)
- `SYS_READC` (0x07)
- `SYS_REMOVE` (0x0e)
- `SYS_RENAME` (0x0f)
- `SYS_SEEK` (0x0a)
- `SYS_SYSTEM` (0x12)
- `SYS_TICKFREQ` (0x31)
- `SYS_TIME` (0x11)
- `SYS_TMPNAM` (0x0d)
- `SYS_WRITE` (0x05)
- `SYS_WRITEC` (0x03)
- `SYS_WRITE0` (0x04)

6.1 SYS_CLOCK (0x10)

Returns the number of centiseconds (hundredths of a second) since the execution started.

Values returned can be of limited use for some benchmarking purposes because of communication overhead or other agent-specific factors. For example, with a debug hardware unit the request is passed back to the host for execution. This can lead to unpredictable delays in transmission and process scheduling.

Use this function to calculate time intervals, by calculating differences between intervals with and without the code sequence to be timed.

6.1.1 Entry

The PARAMETER REGISTER must contain 0. There are no other parameters.

6.1.2 Return

On exit, the RETURN REGISTER contains:

- The number of centiseconds since some arbitrary start point, if the call is successful.
- -1 if the call is not successful. For example, because of a communications error.

Related information

- [SYS_ELAPSED \(0x30\)](#)
- [SYS_TICKFREQ \(0x31\)](#)

6.2 SYS_CLOSE (0x02)

Closes a file on the host system. The handle must reference a file that was opened with [SYS_OPEN \(0x01\)](#).

6.2.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to a one-field argument block:

field 1

Contains a handle for an open file.

6.2.2 Return

On exit, the RETURN REGISTER contains:

- 0 if the call is successful
- -1 if the call is not successful.

Related information

- [SYS_OPEN \(0x01\)](#)

6.3 SYS_ELAPSED (0x30)

Returns the number of elapsed target ticks since execution started.

Use [SYS_TICKFREQ \(0x31\)](#) to determine the tick frequency.

6.3.1 Entry (32-bit)

On entry, the PARAMETER REGISTER points to a two-field data block to be used for returning the number of elapsed ticks:

field 1

The least significant field and is at the low address.

field 2

The most significant field and is at the high address.

6.3.2 Entry (64-bit)

On entry the PARAMETER REGISTER points to a one-field data block to be used for returning the number of elapsed ticks:

field 1

The number of elapsed ticks as a 64-bit value.

6.3.3 Return

On exit:

- On success, the RETURN REGISTER contains 0, the PARAMETER REGISTER is unchanged, and the data block pointed to by the PARAMETER REGISTER is filled in with the number of elapsed ticks.
- On failure, the RETURN REGISTER contains -1, and the PARAMETER REGISTER contains -1.

Note

Some semihosting implementations might not support this semihosting operation, and they always return -1 in the RETURN REGISTER.

Related information

- [SYS_TICKFREQ \(0x31\)](#)

6.4 SYS_ERRNO (0x13)

Returns the value of the C library `errno` variable that is associated with the semihosting implementation.

The `errno` variable can be set by a number of C library semihosted functions, including:

- `SYS_REMOVE` (0x0E)
- `SYS_OPEN` (0x01)
- `SYS_CLOSE` (0x02)
- `SYS_READ` (0x06)
- `SYS_WRITE` (0x05)
- `SYS_SEEK` (0x0A).

Whether `errno` is set or not, and to what value, is entirely host-specific, except where the ISO C standard defines the behavior.

6.4.1 Entry

There are no parameters. The `PARAMETER REGISTER` must be 0.

6.4.2 Return

On exit, the `RETURN REGISTER` contains the value of the C library `errno` variable.

Related information

- `SYS_CLOSE` (0x02)
- `SYS_OPEN` (0x01)
- `SYS_READ` (0x06)
- `SYS_REMOVE` (0x0E)
- `SYS_SEEK` (0x0A)
- `SYS_WRITE` (0x05)

6.5 SYS_EXIT (0x18)

Note

`SYS_EXIT (0x18)` was called `angel_SWIreason_ReportException` in previous versions of the documentation.

An application calls this operation to report an exception to the debugger directly. The most common use is to report that execution has completed, using `ADP_Stopped_ApplicationExit`.

Note

This semihosting operation provides no means for 32-bit callers to indicate an application exit with a specified exit code. Semihosting callers may prefer to check for the presence of the `SH_EXT_EXIT_EXTENDED` extension and use the `SYS_EXIT_EXTENDED (0x20)` operation instead, if it is available.

6.5.1 Entry (32-bit)

On entry, the `PARAMETER` register is set to a reason code describing the cause of the trap. Not all semihosting client implementations will necessarily trap every corresponding event. These reason codes are defined in the following tables.

The following table shows reason codes relating to hardware exceptions. Exception handlers can use these operations to report an exception that has not been handled:

Hardware vector reason codes

Name	Hexadecimal value
<code>ADP_Stopped_BranchThroughZero</code>	<code>0x20000</code>
<code>ADP_Stopped_UndefinedInstr</code>	<code>0x20001</code>
<code>ADP_Stopped_SoftwareInterrupt</code>	<code>0x20002</code>
<code>ADP_Stopped_PrefetchAbort</code>	<code>0x20003</code>
<code>ADP_Stopped_DataAbort</code>	<code>0x20004</code>
<code>ADP_Stopped_AddressException</code>	<code>0x20005</code>
<code>ADP_Stopped_IRQ</code>	<code>0x20006</code>
<code>ADP_Stopped_FIQ</code>	<code>0x20007</code>

The following table shows reason codes relating to software events:

Software reason codes

Name	Hexadecimal value
<code>ADP_Stopped_BreakPoint</code>	<code>0x20020</code>
<code>ADP_Stopped_WatchPoint</code>	<code>0x20021</code>

Name	Hexadecimal value
ADP_Stopped_StepComplete	0x20022
ADP_Stopped_RunTimeErrorUnknown	0x20023
ADP_Stopped_InternalError	0x20024
ADP_Stopped_UserInterruption	0x20025
ADP_Stopped_ApplicationExit	0x20026
ADP_Stopped_StackOverflow	0x20027
ADP_Stopped_DivisionByZero	0x20028
ADP_Stopped_OSSpecific	0x20029

6.5.2 Entry (64-bit)

On entry, the PARAMETER REGISTER contains a pointer to a two-field argument block:

field 1

The exception type, which is one of the set of reason codes in the above tables.

field 2

A subcode, whose meaning depends on the reason code in field 1.

In particular, if field 1 is ADP_Stopped_ApplicationExit then field 2 is an exit status code, as passed to the C standard library `exit()` function. A simulator receiving this request must notify a connected debugger, if present, and then exit with the specified status.

6.5.3 Return

No return is expected from these calls. However, it is possible for the debugger to request that the application continues by performing an RDI_Execute request or equivalent. In this case, execution continues with the registers as they were on entry to the operation, or as subsequently modified by the debugger.

Related information

- [SYS_EXIT_EXTENDED \(0x20\)](#)

6.6 SYS_EXIT_EXTENDED (0x20)

This operation is only supported if the semihosting extension [SH_EXT_EXIT_EXTENDED](#) is implemented.

SH_EXT_EXIT_EXTENDED is reported using feature byte 0, bit 0. If this extension is supported, then the implementation provides a means to report a normal exit with a nonzero exit status in both 32-bit and 64-bit semihosting APIs.

The implementation must provide the semihosting call [SYS_EXIT_EXTENDED \(0x20\)](#) for both A64 and A32/T32 semihosting APIs.

[SYS_EXIT_EXTENDED \(0x20\)](#) is used by an application to report an exception or exit to the debugger directly. The most common use is to report that execution has completed, using `ADP_Stopped_ApplicationExit`.

6.6.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to a two-field argument block:

field 1

The exception type, which should be one of the set of reason codes that are documented for the [SYS_EXIT \(0x18\)](#) call. For example, `ADP_Stopped_ApplicationExit` or `ADP_Stopped_InternalError`.

field 2

A subcode, whose meaning depends on the reason code in field 1. In particular, if field 1 is `ADP_Stopped_ApplicationExit` then field 2 is an exit status code, as passed to the C standard library `exit()` function. A simulator receiving this request must notify a connected debugger, if present, and then exit with the specified status.

6.6.2 Return

No return is expected from these calls.

Note

For the A64 API, this call is identical to the behavior of the mandatory [SYS_EXIT \(0x18\)](#) call. If this extension is supported, then both calls must be implemented.

Related information

- [SYS_EXIT \(0x18\)](#)

6.7 SYS_FLEN (0x0C)

Returns the length of a specified file.

6.7.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to a one-field argument block:

field 1

A handle for a previously opened, seekable file object.

6.7.2 Return

On exit, the RETURN REGISTER contains:

- The current length of the file object, if the call is successful.
- -1 if an error occurs.

6.8 SYS_GET_CMDLINE (0x15)

Returns the command line that is used for the call to the executable, that is, argc and argv.

6.8.1 Entry

On entry, the PARAMETER REGISTER points to a two-field data block to be used for returning the command string and its length:

field 1

A pointer to a buffer of at least the size that is specified in field 2.

field 2

The length of the buffer in bytes.

6.8.2 Return

On exit:

If the call is successful, then the RETURN REGISTER contains 0, the PARAMETER REGISTER is unchanged, and the data block is updated as follows:

field 1

A pointer to a null-terminated string of the command line.

field 2

The length of the string in bytes.

If the call is not successful, then the RETURN REGISTER contains -1.

Note

The semihosting implementation might impose limits on the maximum length of the string that can be transferred. However, the implementation must be able to support a command-line length of at least 80 bytes.

6.9 SYS_HEAPINFO (0x16)

Returns the system stack and heap parameters.

6.9.1 Entry

On entry, the PARAMETER REGISTER contains the address of a pointer to a four-field data block. The contents of the data block are filled by the function. The following C-like pseudocode describes the layout of the block:

```
struct block {  
    void* heap_base;  
    void* heap_limit;  
    void* stack_base;  
    void* stack_limit;  
};
```

6.9.2 Return

On exit, the PARAMETER REGISTER is unchanged and the data block has been updated.

6.10 SYS_ISERROR (0x08)

Determines whether the return code from another semihosting call is an error status or not.

This call is passed a parameter block containing the error code to examine.

6.10.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to a one-field data block:

field 1

The required status word to check.

6.10.2 Return

On exit, the RETURN REGISTER contains:

- 0 if the status field is not an error indication
- A nonzero value if the status field is an error indication.

6.11 SYS_ISTTY (0x09)

Checks whether a file is connected to an interactive device.

6.11.1 *Entry*

On entry, the PARAMETER REGISTER contains a pointer to a one-field argument block:

field 1

A handle for a previously opened file object.

6.11.2 *Return*

On exit, the RETURN REGISTER contains:

- 1 if the handle identifies an interactive device.
- 0 if the handle identifies a file.
- A value other than 1 or 0 if an error occurs.

6.12 SYS_OPEN (0x01)

Opens a file on the host system.

The file path is specified either as relative to the current directory of the host process, or absolute, using the path conventions of the host operating system.

Semihosting implementations must support opening the special path name :semihosting-features as part of the semihosting extensions reporting mechanism. See [Semihosting extensions](#) for details.

ARM targets interpret the special path name :tt as meaning the console input stream, for an open-read or the console output stream, for an open-write. Opening these streams is performed as part of the standard startup code for those applications that reference the C stdio streams.

The semihosting extension SH_EXT_STDOUT_STDERR allows the semihosting caller to open separate output streams corresponding to stdout and stderr. This extension is reported using feature byte 0, bit 1. Use [SYS_OPEN \(0x01\)](#) with the special path name :semihosting-features to access the feature bits. See [Semihosting extensions](#) for details.

If this extension is supported, the implementation must support the following additional semantics to [SYS_OPEN \(0x01\)](#):

- If the special path name :tt is opened with an fopen mode requesting write access (w, wb, w+, or w+b), then this is a request to open stdout.
- If the special path name :tt is opened with a mode requesting append access (a, ab, a+, or a+b), then this is a request to open stderr.

6.12.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to a three-field argument block:

field 1

A pointer to a null-terminated string containing a file or device name.

field 2

An integer that specifies the file opening mode. The *Value of mode* table gives the valid values for the integer, and their corresponding ISO C fopen() mode.

field 3

An integer that gives the length of the string pointed to by field 1.

The length does not include the terminating null character that must be present.

Value of mode

Mode	0	1	2	3	4	5	6	7	8	9	10	11
ISO C fopen mode (Note 1)	r	rb	r+	r+b	w	wb	w+	w+b	a	ab	a+	a+b

Note

1. The non-ANSI option t is not supported.

6.12.2 Return

On exit, the RETURN REGISTER contains:

- A nonzero handle if the call is successful.
- -1 if the call is not successful.

6.13 SYS_READ (0x06)

Reads the contents of a file into a buffer.

The file position is specified either:

- Explicitly by a [SYS_SEEK \(0x0A\)](#).
- Implicitly one byte beyond the previous [SYS_READ \(0x06\)](#) or [SYS_WRITE \(0x05\)](#) request.

The file position is at the start of the file when it is opened, and is lost when the file is closed. Perform the file operation as a single action whenever possible. For example, do not split a read of 16KB into four 4KB chunks unless there is no alternative.

6.13.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to a three-field data block:

field 1

Contains a handle for a file previously opened with [SYS_OPEN \(0x01\)](#).

field 2

Points to a buffer.

field 3

Contains the number of bytes to read to the buffer from the file.

6.13.2 Return

On exit, the RETURN REGISTER contains the number of bytes not filled in the buffer (`buffer_length - bytes_read`) as follows:

- If the RETURN REGISTER is 0, the entire buffer was successfully filled.
- If the RETURN REGISTER is the same as field 3, no bytes were read (EOF can be assumed).
- If the RETURN REGISTER contains a value smaller than field 3, the read succeeded but the buffer was only partly filled. For interactive devices, this is the most common return value.

Related information

- [SYS_READC \(0x07\)](#)

6.14 SYS_READC (0x07)

Reads a byte from the console.

6.14.1 Entry

The PARAMETER REGISTER must contain 0. There are no other parameters or values possible.

6.14.2 Return

On exit, the RETURN REGISTER contains the byte read from the console.

Related information

- [SYS_READ \(0x06\)](#)

6.15 SYS_REMOVE (0x0E)

Deletes a specified file on the host filing system.

6.15.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to a two-field argument block:

field 1

Points to a null-terminated string that gives the path name of the file to be deleted.

field 2

The length of the string.

6.15.2 Return

On exit, the RETURN REGISTER contains:

- 0 if the delete is successful
- A nonzero, host-specific error code if the delete fails.

6.16 SYS_RENAME (0x0F)

Renames a specified file.

6.16.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to a four-field data block:

field 1

A pointer to the name of the old file.

field 2

The length of the old filename.

field 3

A pointer to the new filename.

field 4

The length of the new filename.

Both strings are null-terminated.

6.16.2 Return

On exit, the RETURN REGISTER contains:

- 0 if the rename is successful.
- A nonzero, host-specific error code if the rename fails.

6.17 SYS_SEEK (0x0A)

Seeks to a specified position in a file using an offset specified from the start of the file.

The file is assumed to be a byte array and the offset is given in bytes.

6.17.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to a two-field data block:

field 1

A handle for a seekable file object.

field 2

The absolute byte position to seek to.

6.17.2 Return

On exit, the RETURN REGISTER contains:

- 0 if the request is successful.
- A negative value if the request is not successful. Use [SYS_ERRNO \(0x13\)](#) to read the value of the host `errno` variable describing the error.

Note

The effect of seeking outside the current extent of the file object is undefined.

Related information

- [SYS_ERRNO \(0x13\)](#)

6.18 SYS_SYSTEM (0x12)

Passes a command to the host command-line interpreter.

This enables you to execute a system command such as `dir`, `ls`, or `pwd`. The terminal I/O is on the host, and is not visible to the target.

Caution!

The command that is passed to the host is executed on the host. Ensure that any command passed has no unintended consequences.

6.18.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to a two-field argument block:

field 1

Points to a string to be passed to the host command-line interpreter.

field 2

The length of the string.

6.18.2 Return

On exit, the RETURN REGISTER contains the return status.

6.19 SYS_TICKFREQ (0x31)

Returns the tick frequency.

6.19.1 Entry

The PARAMETER REGISTER must contain 0 on entry to this routine.

6.19.2 Return

On exit, the RETURN REGISTER contains either:

- The number of ticks per second.
- -1 if the target does not know the value of one tick.

Note

Some semihosting implementations might not support this semihosting operation, and they always return -1 in the RETURN REGISTER.

6.20 SYS_TIME (0x11)

Returns the number of seconds since 00:00 January 1, 1970.

This value is real-world time, regardless of any debug agent configuration.

The returned value should be interpreted as unsigned. There is no representation for times before 1970. On 32-bit implementations, this allows times after the year 2038 to be represented, by using values greater than 0x80000000.

6.20.1 Entry

There are no parameters.

6.20.2 Return

On exit, the RETURN REGISTER contains the number of seconds.

6.21 SYS_TMPNAM (0x0D)

Returns a temporary name for a file identified by a system file identifier.

6.21.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to a three-word argument block:

field 1

A pointer to a buffer.

field 2

A target identifier for this filename. Its value must be an integer in the range 0-255.

field 3

Contains the length of the buffer. The length must be at least the value of L_tmpnam on the host system.

6.21.2 Return

On exit, the RETURN REGISTER contains:

- 0 if the call is successful.
- -1 if an error occurs.

The buffer pointed to by the PARAMETER REGISTER contains the filename, prefixed with a suitable directory name.

If you use the same target identifier again, the same filename is returned.

Note

The returned string must be null-terminated.

6.22 SYS_WRITE (0x05)

Writes the contents of a buffer to a specified file at the current file position.

The file position is specified either:

- Explicitly, by a [SYS_SEEK \(0x0A\)](#).
- Implicitly as one byte beyond the previous [SYS_READ \(0x06\)](#) or [SYS_WRITE \(0x05\)](#) request.

The file position is at the start of the file when the file is opened, and is lost when the file is closed.

Perform the file operation as a single action whenever possible. For example, do not split a write of 16KB into four 4KB chunks unless there is no alternative.

6.22.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to a three-field data block:

field 1

Contains a handle for a file previously opened with [SYS_OPEN \(0x01\)](#).

field 2

Points to the memory containing the data to be written.

field 3

Contains the number of bytes to be written from the buffer to the file.

6.22.2 Return

On exit, the RETURN REGISTER contains:

- 0 if the call is successful.
- The number of bytes that are not written, if there is an error.

Related information

- [SYS_WRITE0 \(0x04\)](#)
- [SYS_WRITEC \(0x03\)](#)

6.23 SYS_WRITEC (0x03)

Writes a character byte, pointed to by the PARAMETER REGISTER, to the debug channel.

When executed under an ARM debugger, the character appears on the host debugger console.

6.23.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to the character.

6.23.2 Return

None. The RETURN REGISTER is corrupted.

Related information

- [SYS_WRITE \(0x05\)](#)
- [SYS_WRITE0 \(0x04\)](#)

6.24 SYS_WRITE0 (0x04)

Writes a null-terminated string to the debug channel.

When executed under an ARM debugger, the characters appear on the host debugger console.

6.24.1 Entry

On entry, the PARAMETER REGISTER contains a pointer to the first byte of the string.

6.24.2 Return

None. The RETURN REGISTER is corrupted.

Related information

- [SYS_WRITE \(0x05\)](#)
- [SYS_WRITEC \(0x03\)](#)